https://livecohortbypradeep.com/

DSA Pattern Sheet

https://docs.google.com/document/d/1O_90kJ-3QXDt-fJ9EUBOFJeDaFfLSetV/edit?usp=sharing&ouid=110271307063062639965&rtpof=true&sd=true

Free Masterclass Every Week

I conduct masterclass every week, and join the group for updates.

https://whatsapp.com/channel/0029VbCsEUcGJP8Pca1rZc1o

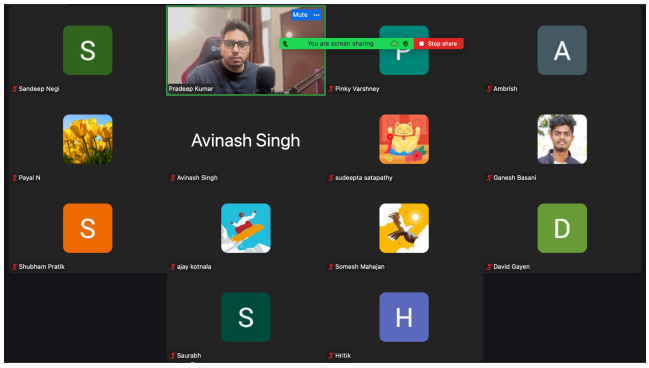
LLD & HLD Free Sheet containing 40+ Questions to Crack System Design Rounds

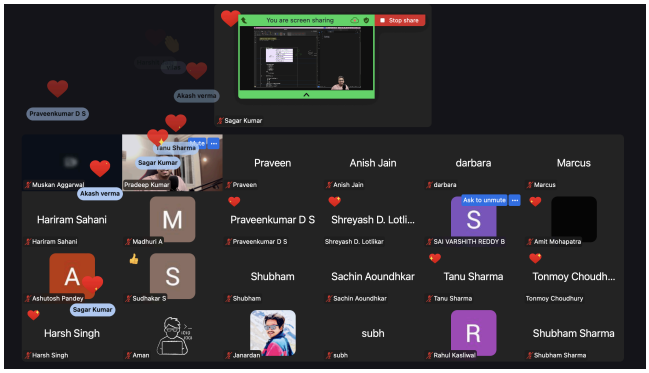
https://docs.google.com/spreadsheets/d/1Qc8R-4PznZyOtgzSas43EKq28cbWcL8H/edit?usp=drive_link&ouid=110271307063062639965&rtpof=true&sd=true

Free LLD Roadmap Below

How to Prepare for Low-Level Design Interviews

https://livecohortbypradeep.com/





Roadmap at a Glance

Phase	Topic	Timeline	Key Areas
Phase 1	OOP Foundations	Week 1	Encapsulation · Abstraction · Inheritance · Polymorphism · UML
Phase 2	SOLID Principles	Week 2	SRP · OCP · LSP · ISP · DIP
Phase 3	Design Patterns	Week 3–4	Creational · Structural · Behavioral
Phase 4	Concurrency in LLD	Week 5	Thread-safety · Locks · Executors · Producer-Consumer
Phase 5	Core LLD Interview Problems	Week 6–8	20+ must-practice problems

Phase 1 OOP Foundations (Week 1)

Must-Know Concepts

- **Encapsulation —** Private state with public behaviour — hide internal details behind a clean API.
- **Abstraction —** Define what an object does (interface) without specifying how it does it.
- **Inheritance —** Model is-a relationships — but prefer composition; avoid deep hierarchies.
- **Polymorphism —** Runtime polymorphism (overriding) is almost always more useful than compile-time.
- **UML Diagrams —** Class diagrams, sequence diagrams — essential for communicating design in interviews.

Interview Expectations

- Assign correct responsibilities to each class
- Use proper access modifiers (private, protected, public)
- Write clean, minimal constructors

Practice Mini-Designs

- Design a Printer
- Design a BankAccount
- Design a Car with Engine abstraction

Phase 2 SOLID Principles (Week 2)

Non-negotiable for SDE-2 interviews. Always explain what problem each principle solves.

SRP	**Single Responsibility Principle**	One class = one reason to change.
OCP	**Open / Closed Principle**	Open for extension, closed for modification.
LSP	**Liskov Substitution Principle**	A child class must be substitutable for its parent without breaking behaviour.
ISP	**Interface Segregation Principle**	Many small, focused interfaces are better than one large, fat interface.
DIP	**Dependency Inversion Principle**	Depend on abstractions (interfaces), not concrete implementations.

Phase 3 Design Patterns (Week 3–4)

Focus on when & why to use each pattern — not memorisation.

💡 **Key Insight: Most LLD problems reduce to Strategy + Factory + Composition**

Creational

- **Singleton (thread-safe) —** Ensure only one instance; use double-checked locking or enum.
- **Factory Method —** Delegate instantiation to subclasses — decouple client from concrete types.
- **Abstract Factory —** Family of related objects without specifying concrete classes.
- **Builder —** Construct complex objects step-by-step; great for config-heavy objects.

Structural

- **Adapter —** Convert an interface into another the client expects.
- **Decorator —** Add responsibilities dynamically without modifying the original class.
- **Facade —** Provide a simplified interface to a complex subsystem.
- **Composite —** Treat individual objects and compositions uniformly (tree structures).

Behavioral (Most Important)

- **Strategy —** Define a family of algorithms and make them interchangeable.
- **Observer —** Notify dependents automatically when state changes (event systems).
- **Command —** Encapsulate requests as objects — supports undo/redo, queuing.
- **State —** Allow an object to alter its behaviour when its state changes.
- **Chain of Responsibility —** Pass a request along a chain of handlers until one handles it.

Phase 4 Concurrency in LLD (Week 5)

SDE-2 interviews often include thread-safety discussion — be prepared.

Must-Know Primitives

- synchronized keyword
- volatile keyword
- Lock & ReentrantLock
- Executors & thread pools
- Thread-safe Singleton
- Producer-Consumer pattern

Apply in Real Designs

- Thread-safe Cache (ConcurrentHashMap + locks)
- Rate Limiter (token bucket, sliding window)
- Parking Lot entry/exit system with concurrent access

Phase 5 Core LLD Interview Problems (Week 6–8)

These are absolute must-designs for any SDE / SDE-2 interview.

Beginner

- Design a Logger
- Design a File System (simplified)
- Design a Vending Machine

Intermediate — SDE-2 Sweet Spot

1. Parking Lot System
2. Elevator (Lift) System
3. LRU Cache
4. Rate Limiter
5. Online Chess Game
6. Splitwise (Expense Sharing System)
7. Movie Ticket Booking System
8. ATM System
9. Notification System
10. File System (Simplified)
11. Ride Booking System (Uber/Ola)
12. URL Shortener
13. Logger System
14. Vending Machine
15. Online Shopping Cart
16. Hotel Booking System
17. Snake and Ladder Game
18. Library Management System
19. Task Scheduler
20. Cache with Expiry (TTL Cache)

Concurrency Concepts

Core Topics

1. Threads & Processes
2. Locks & Mutex
3. Synchronized blocks
4. Deadlock, Livelock, Starvation
5. Race conditions
6. Semaphores
7. wait() / notify() / notifyAll()
8. Thread pools & Executor framework
9. volatile keyword
10. Atomic variables (AtomicInteger etc.)
11. ReentrantLock & ReadWriteLock
12. Producer-Consumer pattern
13. CountDownLatch & CyclicBarrier
14. BlockingQueue
15. CompletableFuture / async patterns

Top 10 Interview Questions

1. Design a **thread-safe Singleton** — explain double-checked locking
2. Implement a **Producer-Consumer** using BlockingQueue
3. How would you design a **rate limiter** that works across threads?
4. What's the difference between `synchronized` and `ReentrantLock`? When do you use each?
5. Design a **connection pool** — handle concurrency for borrow/return
6. Implement a **cache with read-write locks** (readers-writers problem)
7. How do you avoid **deadlock** in a dining philosophers scenario?
8. Design a **task scheduler** using thread pools
9. Implement a **CountDownLatch** from scratch
10. How would you make a **Parking Lot** system thread-safe at scale?

HLD:

1. URL Shortener (TinyURL)
2. Twitter / X Feed
3. WhatsApp Messenger
4. YouTube
5. Uber Ride Sharing
6. Google Drive
7. Rate Limiter

8. Notification System
9. Search Autocomplete
10. BookMyShow

Study Strategy & Tips

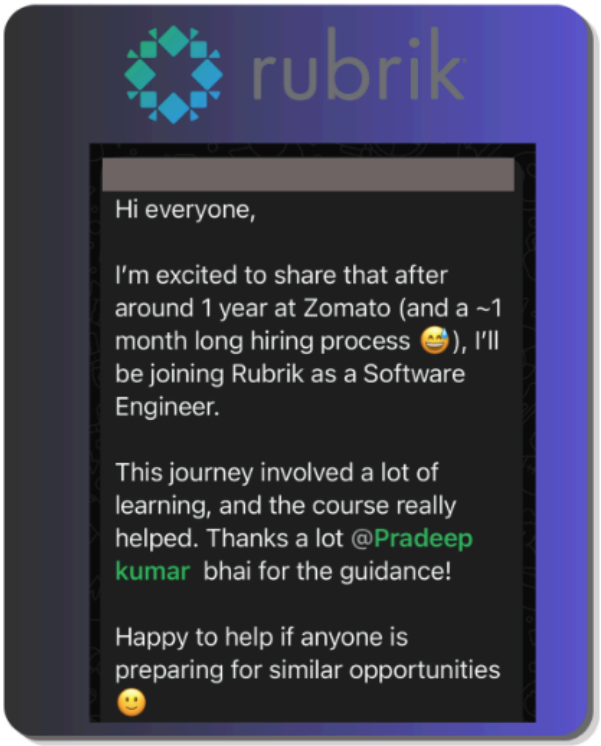
When you first look at the list of LLD problems and expectations, it can feel overwhelming. But once you start practising, you will notice a repeating pattern across all problems. Mastering the core building blocks — OOP, SOLID, and a handful of patterns — unlocks solutions to almost every interview question.

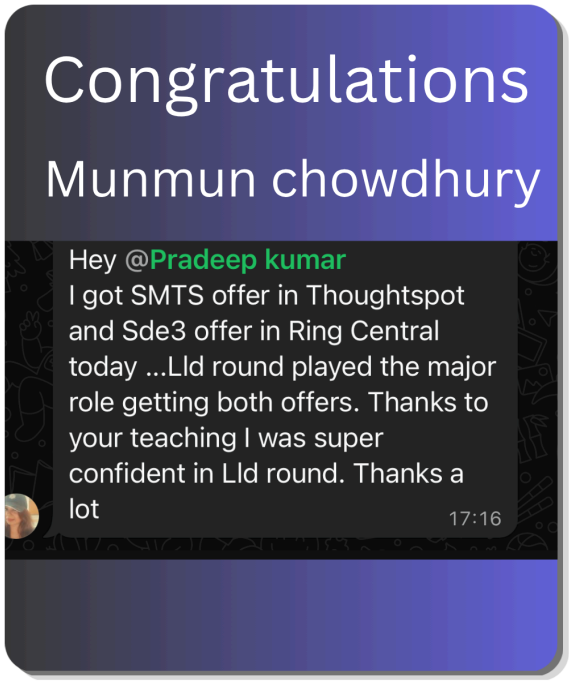
- **Start with why:** Before memorising any pattern, understand the problem it solves. Interviewers reward insight over recall.
- **Code every problem:** Reading is not enough. Write actual classes, run them, refactor them.
- **Time-box phases:** Stick to the weekly schedule; spending too long on basics delays practice time.
- **Explain out loud:** In interviews, narrate your thinking. A good explanation with a decent design beats a great design explained poorly.
- **Review real code:** Read open-source implementations of patterns (e.g., Java's java.util.* library is full of examples).

What Students Say about Live Cohort (Testimonials)

> *"The way you have adopted to make us understand seems to be the best way."*
>
> *"The deep dives into SOLID principles and Design Patterns using real-world examples like a Parking Lot and Elevator System made complex concepts feel intuitive."*
>
> *"I'm having a great learning experience. Your structured and well-organized approach to explaining LLD makes complex concepts easy to understand and brings great clarity."*
>
> *"Complex design principles were explained clearly with practical examples. I now feel much more confident writing clean, scalable, and maintainable code."*
>
> *"Very good and recommended to everyone who place importance on knowledge over money in the IT industry."*
>
> *"The structure of the program is well designed, and the sessions are interactive and engaging, which makes it easy to stay focused and involved."*
>
> *"Pradeep is very approachable and always easy to reach out to for any doubts or clarifications, which creates a comfortable and supportive learning environment."*

Students Success Stories:




Congratulations
Munmun chowdhury
Hey @Pradeep kumar
I got SMTS offer in Thoughtspot and Sde3 offer in Ring Central today ...Lld round played the major role getting both offers. Thanks to your teaching I was super confident in Lld round. Thanks a lot
17:16

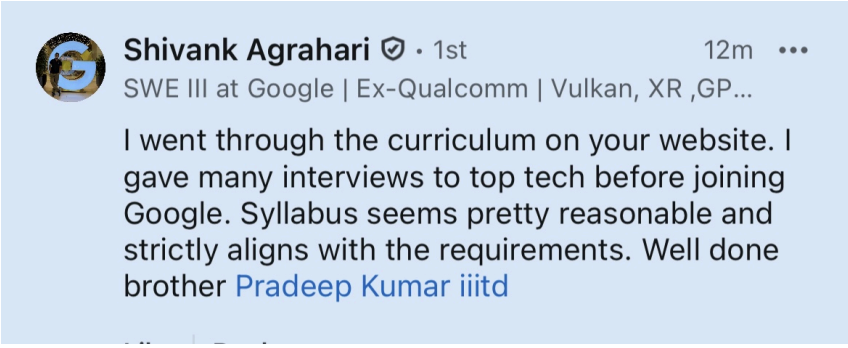

60% HIKE
Hi @Pradeep kumar
I am very happy to share that after a struggle of 1 year I finally cracked a good company and got a 60% hike.
Thanks a lot Pradeep bhai your course helped a lot.
19:32
8

Resources & Links

- **Claude Prompt:**
https://docs.google.com/document/d/1xVFf9jdz0TUFcbnzUFWk6K0T8I1gTbsgKrP5F7Yxuu8/edit
- **Free LLD Masterclass Registration** https://forms.gle/NCw4YfhSYvSosxvC7
- **Live LLD Cohort Enrollment** https://livecohortbypradeep.com/
- **Instagram — LLD Content** https://www.instagram.com/pradeep_kumar_iiitd/
- **Microsoft SDE-2 Interview Experience (LeetCode)**
https://leetcode.com/discuss/post/7545165/microsoft-sde-2-l6162-interview-experien-e5q7/

Enroll For a Live Coding Experience of Low Level Design and Crack Interviews

https://livecohortbypradeep.com/



DSA Company-wise Questions

https://lcpremium.tiiny.site/

Claude Prompt

https://docs.google.com/document/d/1xVFf9jdz0TUFcbnzUFWk6K0T8I1gTbsgKrP5F7Yxuu8/edit